LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 – INHERITANCE



Khairun Fahmi

103132400035

S1SD-05-A

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Inheritance adalah pilar dasar pemrograman berorientasi objek yang memungkinkan sebuah class (subclass) mewarisi atribut dan method dari class lain (superclass) melalui keyword `extends`, sehingga duplikasi kode dapat dihindari dan class tersusun secara hierarkis dari yang umum ke yang lebih spesifik dengan hubungan “is-a”, termasuk pewarisan bertingkat meskipun Java hanya mendukung single inheritance. Keyword `super` digunakan oleh subclass untuk memanggil constructor superclass (`super(...)`, yang harus berada di baris pertama constructor), method superclass, maupun atributnya. Overloading adalah pembuatan beberapa method bernama sama dalam satu class dengan jumlah atau tipe parameter berbeda yang ditentukan saat kompilasi (compile-time polymorphism), sedangkan overriding adalah penulisan ulang method superclass oleh subclass dengan nama, parameter, dan return type yang sama sehingga versi subclass yang dijalankan saat program berjalan. Polymorphism memungkinkan variabel bertipe superclass menyimpan objek subclass dan memanggil method yang di-override sesuai objek aslinya (dynamic binding), meskipun method yang hanya dimiliki subclass memerlukan casting agar dapat dipanggil. Seluruh konsep ini didukung oleh encapsulation, yaitu pembatasan akses atribut dengan modifier seperti `private` yang diakses lewat getter atau constructor superclass, di mana constructor sendiri tidak diwariskan sehingga constructor subclass wajib memanggil constructor superclass baik secara otomatis maupun eksplisit.

B. Guided

1. Inheritance

A

```
package guidedmodul06;

public class A {
    int x;
    int y;

    void tampilXY() {
        System.out.println("nilai x: " + x + " nilai y: " + y);
    }
}
```

B

```
package guidedmodul06;

public class B extends A {
    int z;

    void jumlahXY() {
        System.out.println("jumlah: " + (x + y + z));
```

```
    }
}
```

DemoInheritance

```
package guidedmodul06;

public class DemoInheritance {
    public static void main(String[] args) {
        A superclass = new A();
        B subclass = new B();

        System.out.println("superclass :");
        superclass.x = 3;
        superclass.y = 4;
        superclass.tampilXY();

        System.out.println("subclass :");
        // member superclass dapat diakses dari subclassnya
        subclass.x = 1;
        subclass.y = 2;
        subclass.tampilXY();

        // member tambahan hanya ada di subclass
        subclass.z = 5;
        subclass.jumlahXY();
    }
}
```

Output :

```
superclass :
nilai x: 3 nilai y: 4
subclass :
nilai x: 1 nilai y: 2
jumlah: 8
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

2. Keyword Super

Employee

```
package guidedmodul06;

public class Employee {
    private String name;
    String departemen;

    public Employee(String s) {
        name = s;
```

```
    }

    public void tampilNama() {
        System.out.println("nama : " + name);
    }
}
```

Manager

```
package guidedmodul06;

public class Manager extends Employee {
    private String alamat;

    public Manager(String nama, String s) {
        /* memanggil konstruktor employee */
        super(nama);
        alamat = s;
    }

    public void tampilAlamat() {
        /* menginisialisasi variabel departemen yang ada pada superclass */
        super.departemen = "Personalia";

        /* memanggil method tampilNama() yang ada pada superclass */
        super.tampilNama();

        /* menampilkan variabel departemen yang telah diinisialisasi */
        System.out.println("alamat : " + alamat);
        System.out.println("departemen : " + super.departemen);
    }

    public static void main(String[] args) {
        /* membuat objek */
        Manager adi = new Manager("adi", "sukabirus");
        adi.tampilAlamat();
    }
}
```

Output :

```
nama : adi
alamat : sukabirus
departemen : Personalia
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

3. Overload Method

Lagu

```
package guidedmodul06;

public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }

    void CetakKeLayar() {
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}
```

DemoOver2

```
package guidedmodul06;

public class DemoOver2 {
    public static void main(String[] args) {
        Lagu d, e;
        d = new Lagu();
        e = new Lagu();

        d.IsiParam("Lagu 1");
        e.IsiParam("kepastian yang kutunggu", "GiGi");

        d.CetakKeLayar();
        e.CetakKeLayar();
    }
}
```

Output:

```
exec:3.0.3:exec (default-cli) @ GuidedModul06
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : kepastian yang kutunggu, pencipta : GiGi
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

4. Overriding Method

A2

```
package guidedmodul06;

public class A2 {
    public void tampilkanKeLayar() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

B2

```
package guidedmodul06;

public class B2 extends A2 {
    @Override
    public void tampilkanKeLayar() {
        super.tampilkanKeLayar(); // method milik superclass dipanggil
        System.out.println("Method milik class B dipanggil...");
    }
}
```

DemoOverriding

```
package guidedmodul06;

public class DemoOverriding {
    public static void main(String[] args) {
        B2 subOb = new B2();
        subOb.tampilkanKeLayar();
    }
}
```

Output

```
Method milik class A dipanggil...
Method milik class B dipanggil...
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

5. Latihan

Lingkaran

```
package guidedmodul06;

public class Lingkaran {
    private int jarijari;

    public void setJarijari(int jarijari) {
        this.jarijari = jarijari;
    }

    public int getJarijari() {
        return jarijari;
    }

    public int getLuas() {
        return (int) (Math.PI * jarijari * jarijari);
    }

    // OVERLOAD: nama sama, tetapi menerima parameter
    public int getLuas(int jarijari) {
        return (int) (Math.PI * jarijari * jarijari);
    }
}
```

Bola

```
package guidedmodul06;

public class Bola extends Lingkaran {
    private int jarijari;

    public Bola(int jarijari) {
        this.jarijari = jarijari;
        super.setJarijari(jarijari); // atribut private di superclass diisi lewat
setter
    }
```

```
    // OVERRIDE: luas lingkaran diganti menjadi luas permukaan bola
    @Override
    public int getLuas() {
        return (int) (4 * Math.PI * getJarijari() * getJarijari());
    }

    public int getVolume() {
        return (int) (4.0 / 3 * Math.PI * Math.pow(getJarijari(), 3));
    }
}
```

DemoLatihan

```
package guidedmodul06;

public class DemoLatihan {
    public static void main(String[] args) {
        Lingkaran lingkaran = new Lingkaran();
        lingkaran.setJarijari(7);
        System.out.println("=== Lingkaran ===");
        System.out.println("Jari-jari       : " + lingkaran.getJarijari());
        System.out.println("Luas            : " + lingkaran.getLuas());
        System.out.println("Luas (jari 10)  : " + lingkaran.getLuas(10));

        Bola bola = new Bola(7);
        System.out.println("=== Bola ===");
        System.out.println("Jari-jari       : " + bola.getJarijari());
        System.out.println("Luas permukaan  : " + bola.getLuas());
        System.out.println("Volume          : " + bola.getVolume());
    }
}
```

Output:

```
--- exec:3.6.3:exec (default-cli) @ GuidedModul06 ---
=== Lingkaran ===
Jari-jari       : 7
Luas            : 153
Luas (jari 10)  : 314
=== Bola ===
Jari-jari       : 7
Luas permukaan  : 615
Volume          : 1436
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

C. Unguided

1. Soal Unguided

Sebuah tim *machine learning* ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis *deep learning* yang merupakan pengembangan dari model klasifikasi biasa.

Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

Jawaban :

ModelML

```
package com.mycompany.modelml;

public class ModelML {
    private String nama;
    private int jumlahData;

    public ModelML(String nama, int jumlahData) {
        this.nama = nama;
        this.jumlahData = jumlahData;
    }

    // cara bagi kelas lain membaca jumlah data latih
    public int getJumlahData() {
        return jumlahData;
    }

    public void tampilkanInfo() {
        System.out.println("Model: " + nama + " | Data latih: " + jumlahData);
    }

    // OVERLOADING 1: tanpa parameter
    public int hitungBiaya() {
        return jumlahData * 2;
    }

    // OVERLOADING 2: dengan parameter epoch
    public int hitungBiaya(int epoch) {
        return jumlahData * epoch;
    }
}
```

ModelRegresi

```
package com.mycompany.modelml;

public class ModelRegresi extends ModelML {
    private int jumlahFitur;
```

```
    public ModelRegresi(String nama, int jumlahData, int jumlahFitur) {
        super(nama, jumlahData);
        this.jumlahFitur = jumlahFitur;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    // OVERRIDE: biaya induk + (fitur x 10)
    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    // OVERLOADING baru: epoch dan ukuran batch
    public int hitungBiaya(int epoch, int ukuranBatch) {
        return getJumlahData() * epoch / ukuranBatch;
    }
}
```

ModelKlasifikasi

```
        package com.mycompany.modelml;

public class ModelKlasifikasi extends ModelML {
    private int jumlahKelas;

    public ModelKlasifikasi(String nama, int jumlahData, int jumlahKelas) {
        super(nama, jumlahData);
        this.jumlahKelas = jumlahKelas;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    // OVERRIDE: biaya induk x jumlah kelas
    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }

    // hitungBiaya(int epoch) TIDAK di-override
}
```

ModelDeepLearning

```
package com.mycompany.modelml;

public class ModelDeepLearning extends ModelKlasifikasi {
   private int jumlahLayer;

   public ModelDeepLearning(String nama, int jumlahData, int jumlahKelas, int
jumlahLayer) {
      super(nama, jumlahData, jumlahKelas);
      this.jumlahLayer = jumlahLayer;
   }

   @Override
   public void tampilkanInfo() {
      super.tampilkanInfo();
      System.out.println("Jumlah layer: " + jumlahLayer);
   }

   // OVERRIDE: biaya ModelKlasifikasi + (layer x 200)
   @Override
   public int hitungBiaya() {
      return super.hitungBiaya() + (jumlahLayer * 200);
   }
}
```

Main

```
package com.mycompany.modelml;

public class Main {
   public static void main(String[] args) {
      // satu array bertipe kelas induk (polymorphism)
      ModelML[] daftarModel = {
         new ModelRegresi("Regresi Linier", 500, 4),
         new ModelKlasifikasi("Decision Tree", 800, 3),
         new ModelDeepLearning("Neural Network", 1000, 5, 3)
      };

      int totalBiaya = 0;

      for (ModelML m : daftarModel) {
         m.tampilkanInfo();
         int biaya = m.hitungBiaya();
         System.out.println("Biaya komputasi: " + biaya);
         System.out.println("---");
         totalBiaya += biaya;
```

```
        }

        System.out.println("Total biaya: " + totalBiaya);

        // biaya 10 epoch Neural Network (elemen ke-2 array)
        System.out.println("Biaya 10 epoch (Neural Network): "
                + daftarModel[2].hitungBiaya(10));

        // biaya 10 epoch Regresi Linier (elemen ke-0 array)
        System.out.println("Biaya 10 epoch (Regresi Linier): "
                + daftarModel[0].hitungBiaya(10));

        // method 2 parameter hanya ada di ModelRegresi, jadi perlu casting
        ModelRegresi regresi = (ModelRegresi) daftarModel[0];
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): "
                + regresi.hitungBiaya(10, 50));
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ModelML ---
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Penjelasan :

Program unguided ini memodelkan perbandingan estimasi biaya komputasi tiga jenis model machine learning dengan memanfaatkan konsep inheritance, overloading, overriding, dan polymorphism. Kelas induk ModelML menyimpan atribut private nama dan jumlahData, menyediakan getter getJumlahData() agar bisa dibaca kelas turunan, method tampilkanInfo(), serta dua method hitungBiaya() yang merupakan

overloading (tanpa parameter menghasilkan jumlahData * 2, dengan parameter epoch menghasilkan jumlahData * epoch). Kelas ModelRegresi mewarisi ModelML, menambah atribut jumlahFitur, memanggil constructor induk lewat super(nama, jumlahData), meng-override tampilkanInfo() dan hitungBiaya() (biaya induk ditambah jumlahFitur * 10), serta menambah versi ketiga hitungBiaya(int epoch, int ukuranBatch) sehingga objeknya punya tiga versi penghitung biaya. Kelas ModelKlasifikasi mewarisi ModelML, menambah atribut jumlahKelas, dan meng-override hitungBiaya() menjadi biaya induk dikali jumlahKelas, sedangkan hitungBiaya(int epoch) sengaja tidak di-override sehingga memakai versi induk. Kelas ModelDeepLearning mewarisi ModelKlasifikasi (bukan langsung ModelML) sehingga membentuk pewarisan bertingkat, menambah atribut jumlahLayer, dan meng-override hitungBiaya() menjadi biaya ModelKlasifikasi ditambah jumlahLayer * 200. Pada class Main, ketiga objek disimpan dalam satu array bertipe ModelML (polymorphism), lalu perulangan memanggil tampilkanInfo() dan hitungBiaya() pada tiap elemen, di mana versi method yang dijalankan mengikuti tipe objek aslinya, dan hasilnya dijumlahkan menjadi total biaya 16440 (1040 + 4800 + 10600). Terakhir, hitungBiaya(10) dipanggil untuk Neural Network (10000) dan Regresi Linier (5000), sedangkan hitungBiaya(10, 50) (100) memerlukan casting ke ModelRegresi karena method tersebut hanya dimiliki kelas itu, bukan kelas induk.

D. Kesimpulan

Berdasarkan seluruh praktikum Modul 6, dapat disimpulkan bahwa inheritance memungkinkan sebuah class (subclass) mewarisi atribut dan method dari class lain (superclass) melalui keyword extends, sehingga duplikasi kode dapat dihindari dan hubungan antar class tersusun secara hierarkis dari yang umum ke yang lebih spesifik. Pada bagian guided, hal ini terlihat pada class B yang dapat memakai x, y, dan tampilXY() milik A, serta pada Manager yang memakai super(nama) untuk memanggil constructor Employee dan super.tampilNama() untuk mengakses member superclass. Konsep overloading terbukti memungkinkan beberapa method bernama sama dalam satu class selama jumlah atau tipe parameternya berbeda, seperti IsiParam() pada class Lagu dan getLuas() pada Lingkaran, sedangkan overriding memungkinkan subclass menulis ulang method milik superclass dengan signature yang sama, seperti tampilkanKeLayar() pada class B2 dan getLuas() pada Bola. Pada soal unguided, seluruh konsep tersebut diterapkan dalam pewarisan bertingkat (ModelML → ModelKlasifikasi → ModelDeepLearning) dengan atribut yang seluruhnya private, sehingga akses data antar class dilakukan melalui getter dan pemanggilan super. Penerapan polymorphism lewat array bertipe ModelML menunjukkan bahwa satu perulangan dapat memanggil method yang sama namun menghasilkan perilaku berbeda sesuai objek aslinya, dengan total biaya 16440 yang sesuai contoh soal. Dengan demikian, inheritance, overloading, overriding, dan polymorphism terbukti saling melengkapi untuk

membuat program berorientasi objek yang lebih ringkas, terstruktur, mudah dikembangkan, dan mudah dipelihara.